MILESTONE 2 - NARRATIVE FLOW CO WRITER
MODULE 2 - BACK END

EXECUTIVE SUMMARY

NFCW is a local-first AI storytelling system that runs entirely offline. It integrates Ollama as the neural inference engine and Streamlit as the interactive frontend. The system enables real-time narrative generation, persistent story memory, and professional export capabilities without external API dependencies, ensuring privacy, performance, and zero operational cost.

I. SYSTEM ARCHITECTURE OVERVIEW

The system operates using a three-layer architecture.

Layer	Component	Technology	Function
Interface Layer	Frontend	Streamlit	UI rendering and interaction
Orchestration Layer	Backend Logic	Python	State management and control
Neural Engine Layer	AI Engine	Ollama + Llama 3	Local LLM inference

Communication occurs through: http://127.0.0.1:11434

II. WINDOWS INSTALLATION AND SETUP PROCEDURE

Follow the steps below in order.

Step 1 — Install Ollama Engine

Download and run: OllamaSetup.exe

This installs:

- Ollama CLI
- Background inference service
- System PATH integration
- *Verification:* Ollama icon visible in Windows System Tray

Step 2 — Download Llama 3 Model
Open PowerShell and run:
ollama pull llama3
This downloads the model (~4.7 GB) to: %HOMEPATH%\.ollama\models

Step 3 — Navigate to Project Directory
Open PowerShell and run:
cd path\to\your\project

Step 4 — Create Python Virtual Environment
Run:
python -m venv venv

Step 5 — Activate Virtual Environment
Run:
.\venv\Scripts\activate
Terminal will display: (venv)

Step 6 — Install Required Dependencies
Run:
pip install streamlit ollama python-docx

Step 7 — Start Ollama Neural Engine
Run:
ollama run llama3

Step 8 — Launch Narrative Sanctuary Application
Open new PowerShell window. Navigate to project folder, activate environment, and run:
streamlit run app.py

Step 9 — Access Application Interface
Open browser and go to: http://localhost:3000

Step 10 – Usability

III. NEURAL INFERENCE PIPELINE

Execution flow:

- User inputs narrative text
- Python captures context via session_state
- Request sent to Ollama service
- Llama 3 performs inference
- Response streamed to Streamlit interface
- Story saved automatically

Communication endpoint: http://127.0.0.1:11434

IV. STATE MANAGEMENT AND DATA PERSISTENCE

- **Active Manuscript:** st.session_state.messages
- **Persistent storage file:** chat_history.json
- **Export formats supported:** TXT, DOCX
- **Storage location:** Project directory or Windows Downloads folder

---

V. PERFORMANCE REQUIREMENTS

Component	Minimum Requirement	Note
RAM	8 GB	Llama 3 8B requires ~5GB of VRAM/RAM
Storage	SSD recommended	NVMe reduces model load times
CPU	Multi-core processor	Necessary for handling background threads
GPU	NVIDIA or AMD	Automatically detected by Ollama for acceleration
Network	Port 11434 open	Must be open for local loopback

VI. CODE OPTIMIZATION FEATURES

- **Sliding Context Window:** Limits context to recent messages using chat_obj["messages"][-6:]. This ensures faster inference and reduced memory usage.

- **Persistent Storage System:** Automatic save to chat_history.json prevents data loss.
- **Document Export Engine:** Uses in-memory generation via BytesIO() for professional manuscript export.

VII. TROUBLESHOOTING PROCEDURE

- **Problem:** Ollama Not Running -> **Solution:** ollama run llama3
- **Problem:** Model Not Installed -> **Solution:** ollama pull llama3
- **Problem:** Virtual Env Not Activated -> **Solution:** .\venv\Scripts\activate
- **Problem:** Streamlit Not Starting -> **Solution:** streamlit run app.py
- **Problem:** Corrupted Chat History -> **Solution:** Delete chat_history.json

---

VIII. COMPLETE COMMAND REFERENCE

- OllamaSetup.exe
- ollama pull llama3
- cd path\to\your\project
- python -m venv venv
- .\venv\Scripts\activate
- pip install streamlit ollama python-docx
- ollama run llama3
- streamlit run app.py

IX. SYSTEM CAPABILITIES

- Fully offline AI inference
- Real-time narrative generation
- Persistent story memory
- Professional manuscript export
- Zero API cost operation
- Complete local privacy

X. CONCLUSION

Narrative Sanctuary transforms a standard Windows system into a fully autonomous AI storytelling workstation. By hosting Llama 3 locally and orchestrating inference through Python and Streamlit, the system achieves production-grade performance without external dependencies.

This architecture establishes a private, scalable, and cinematic AI creative partner capable of real-time narrative collaboration.

By
R.V. Aswath Narayanan
aswathnarayanan48@gmail.com